

Analiza Algoritmilor

NP-hard & NP-complete problems

Matei Popovici

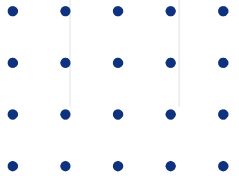


We often use algorithms for existing problems to solve new problems.

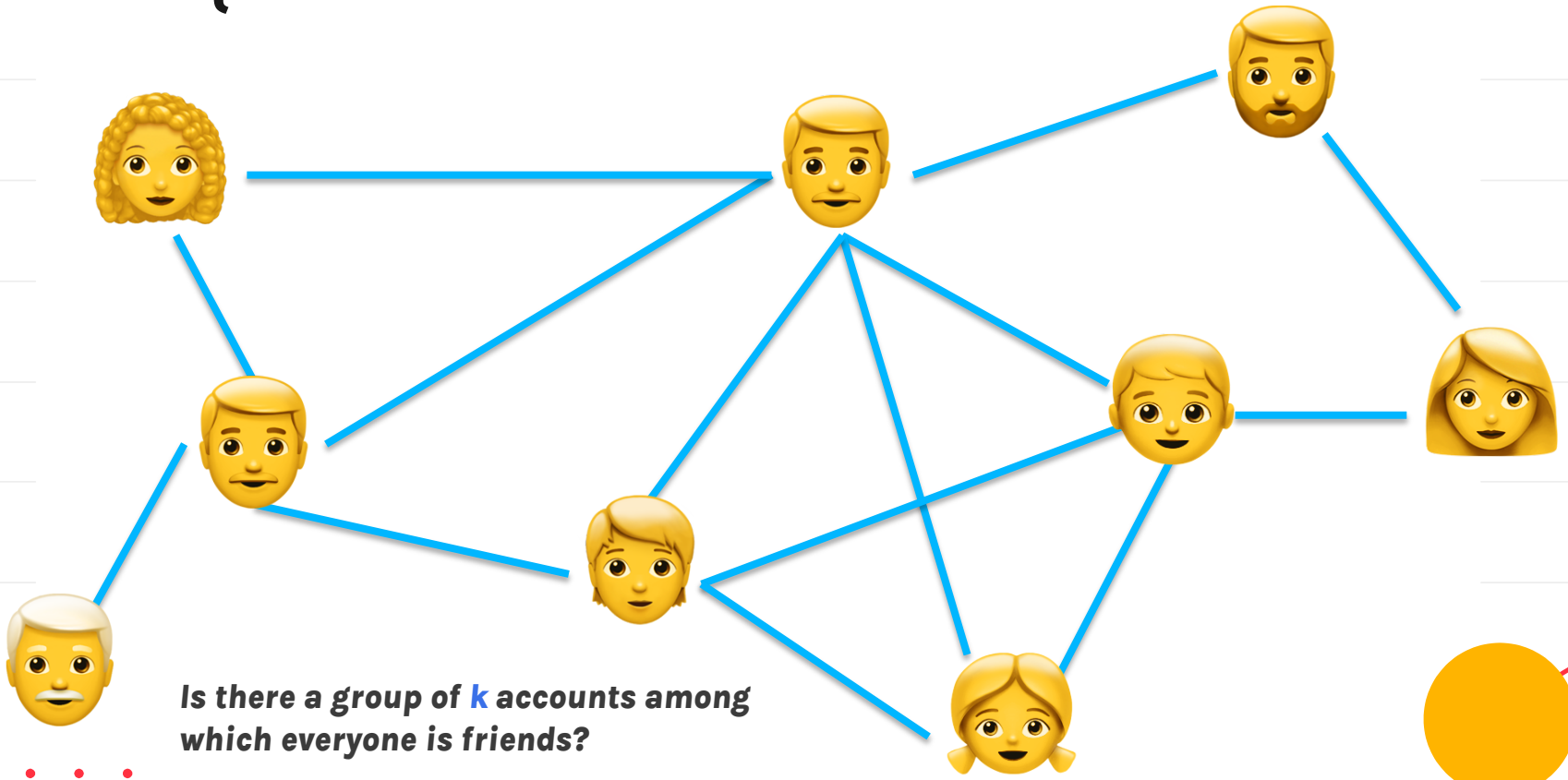
isPalindrome(L) = reverse(L) == L

max(L) = sort(last(L))

k-clique(G,k) = k-Vertex-Cover(complement(G),|V|-k)

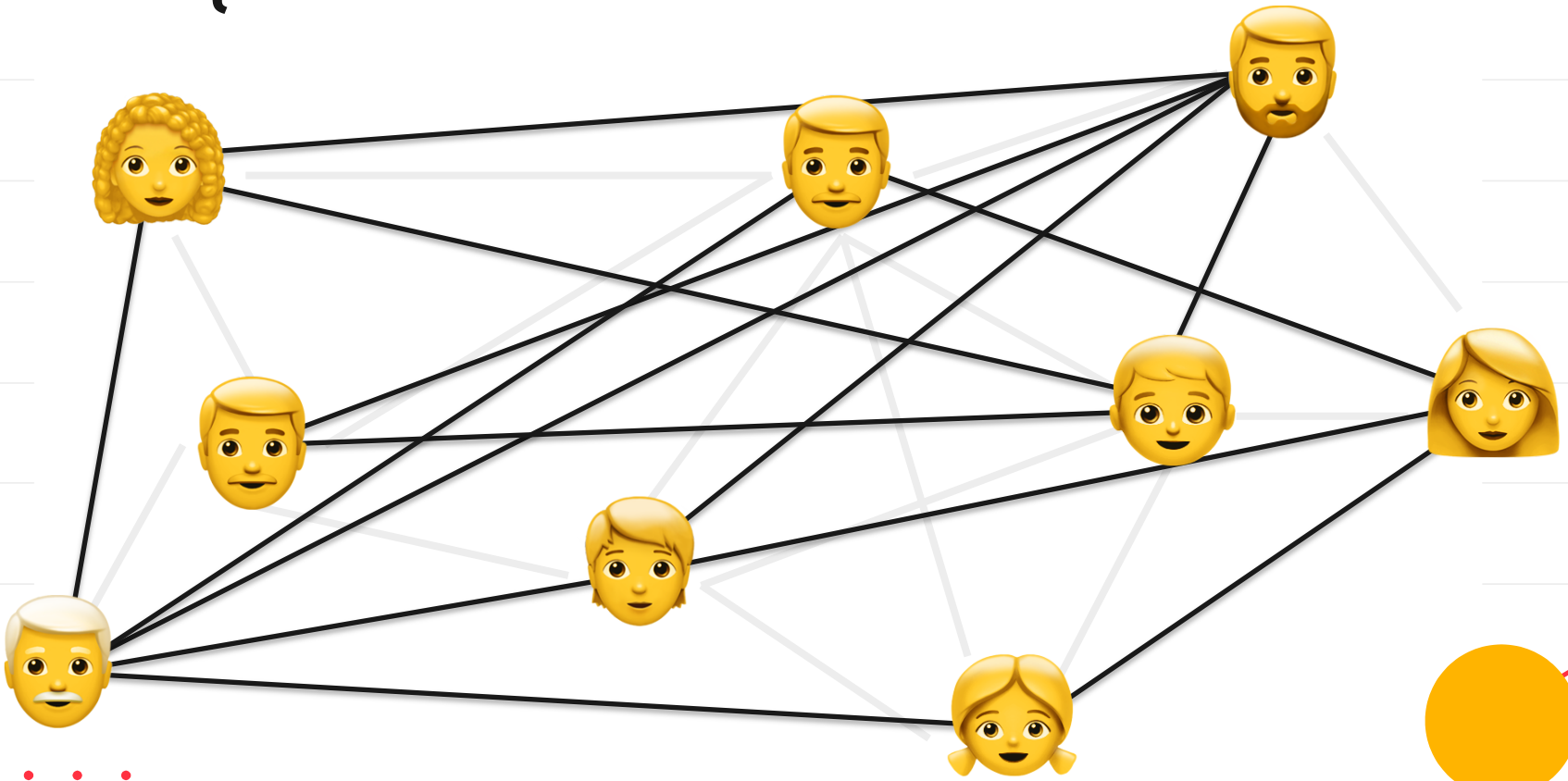


K-Clique reduction to k-Vertex-Cover

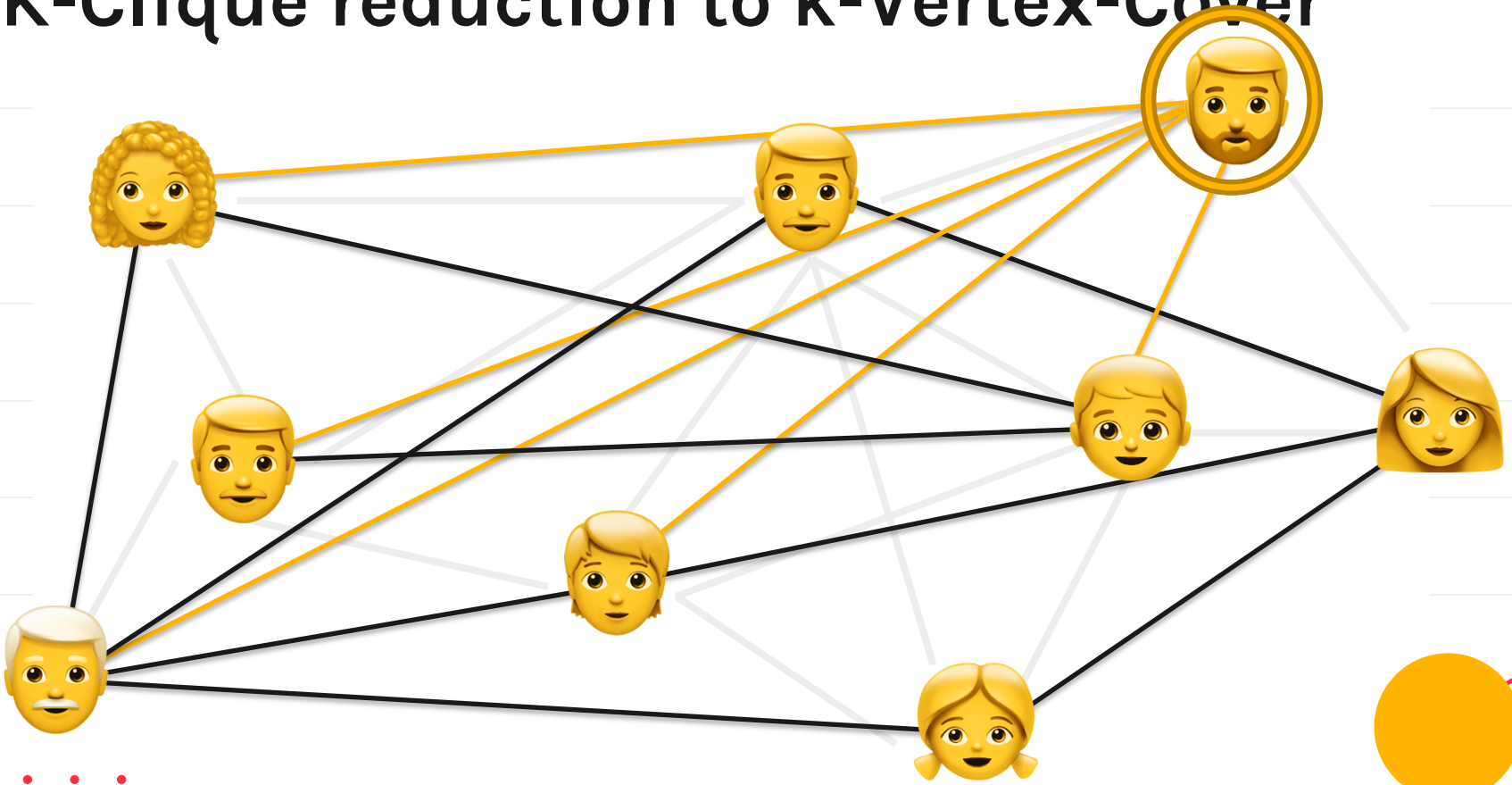


Is there a group of k accounts among which everyone is friends?

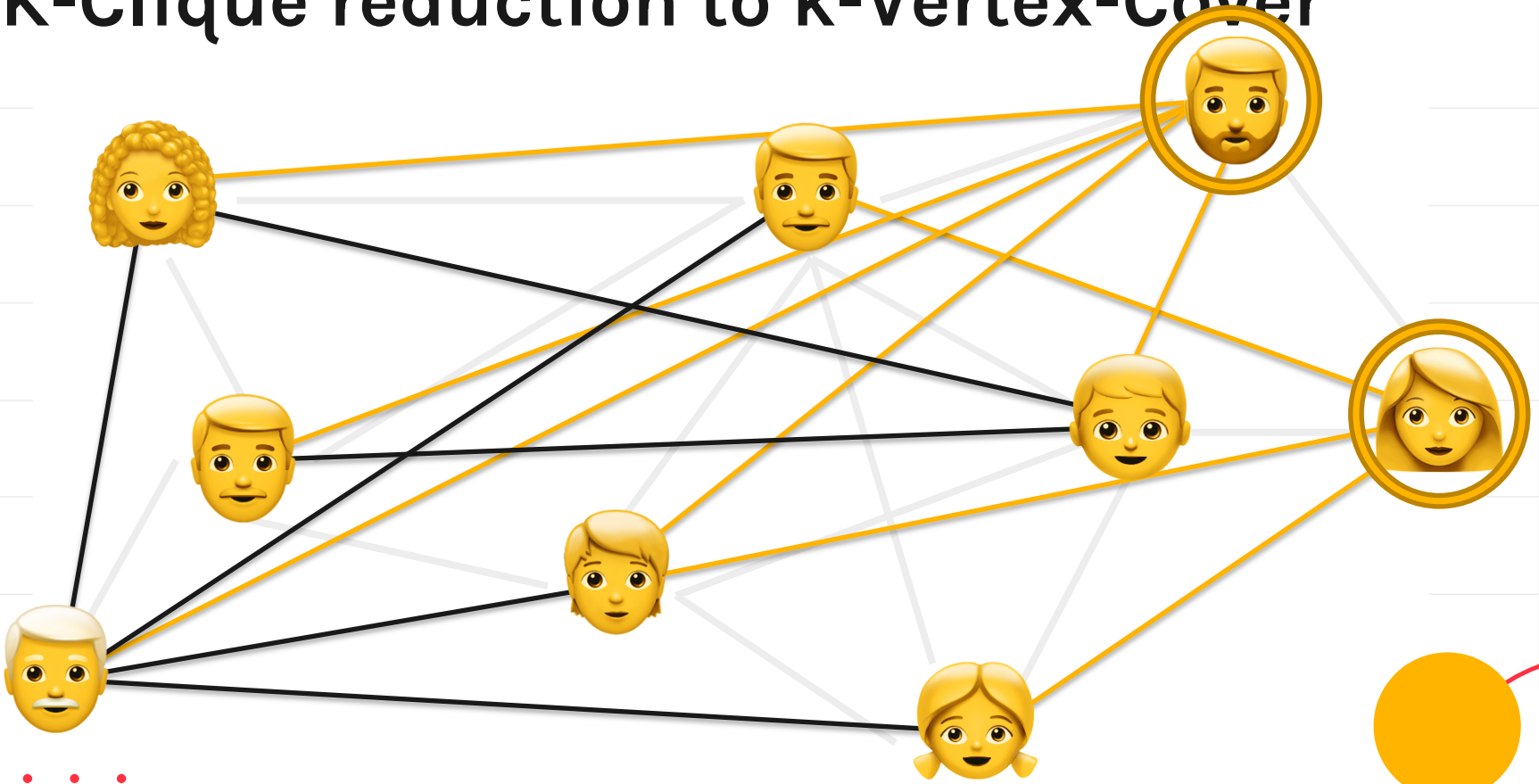
K-Clique reduction to k-Vertex-Cover



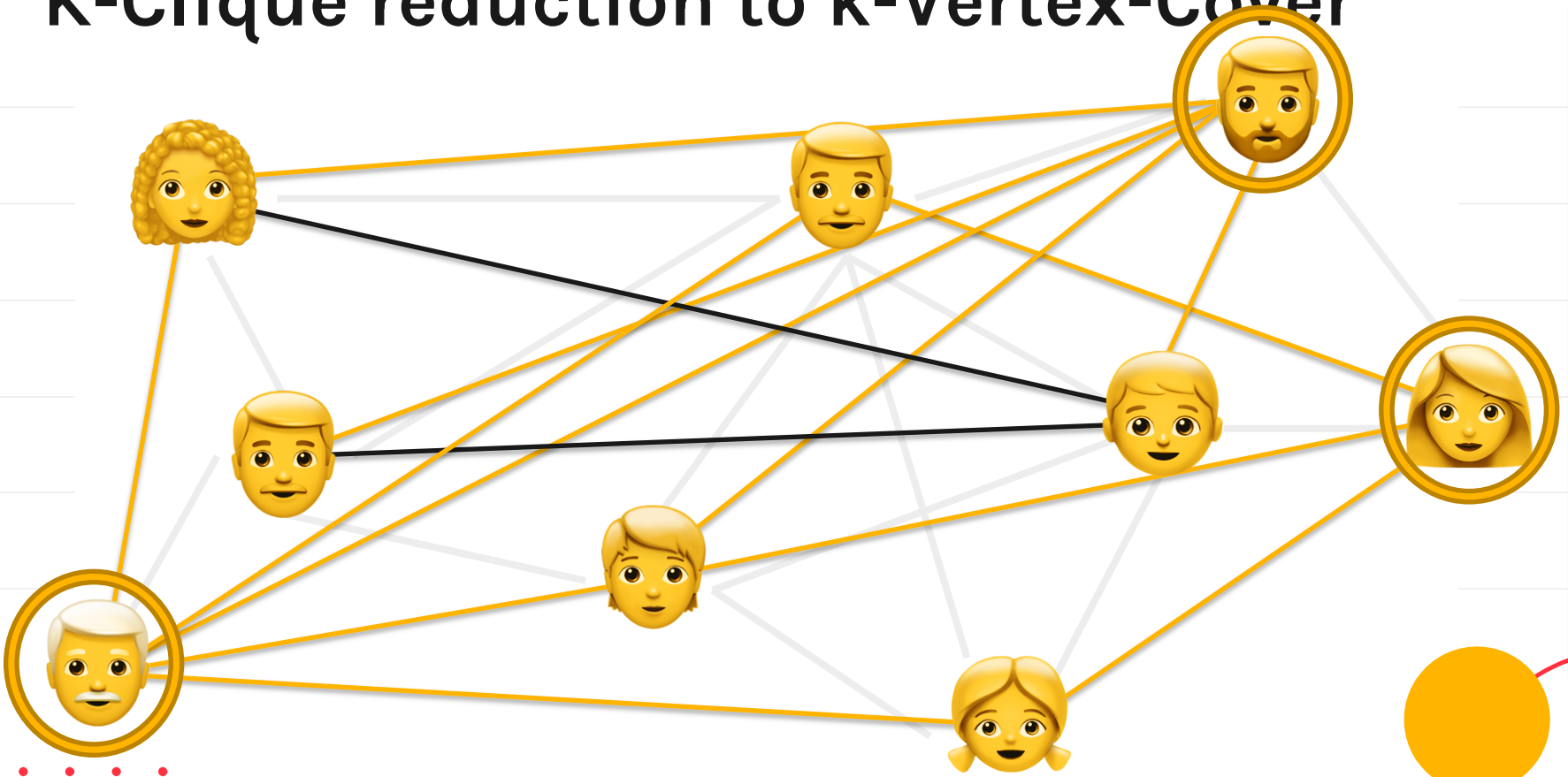
K-Clique reduction to k-Vertex-Cover



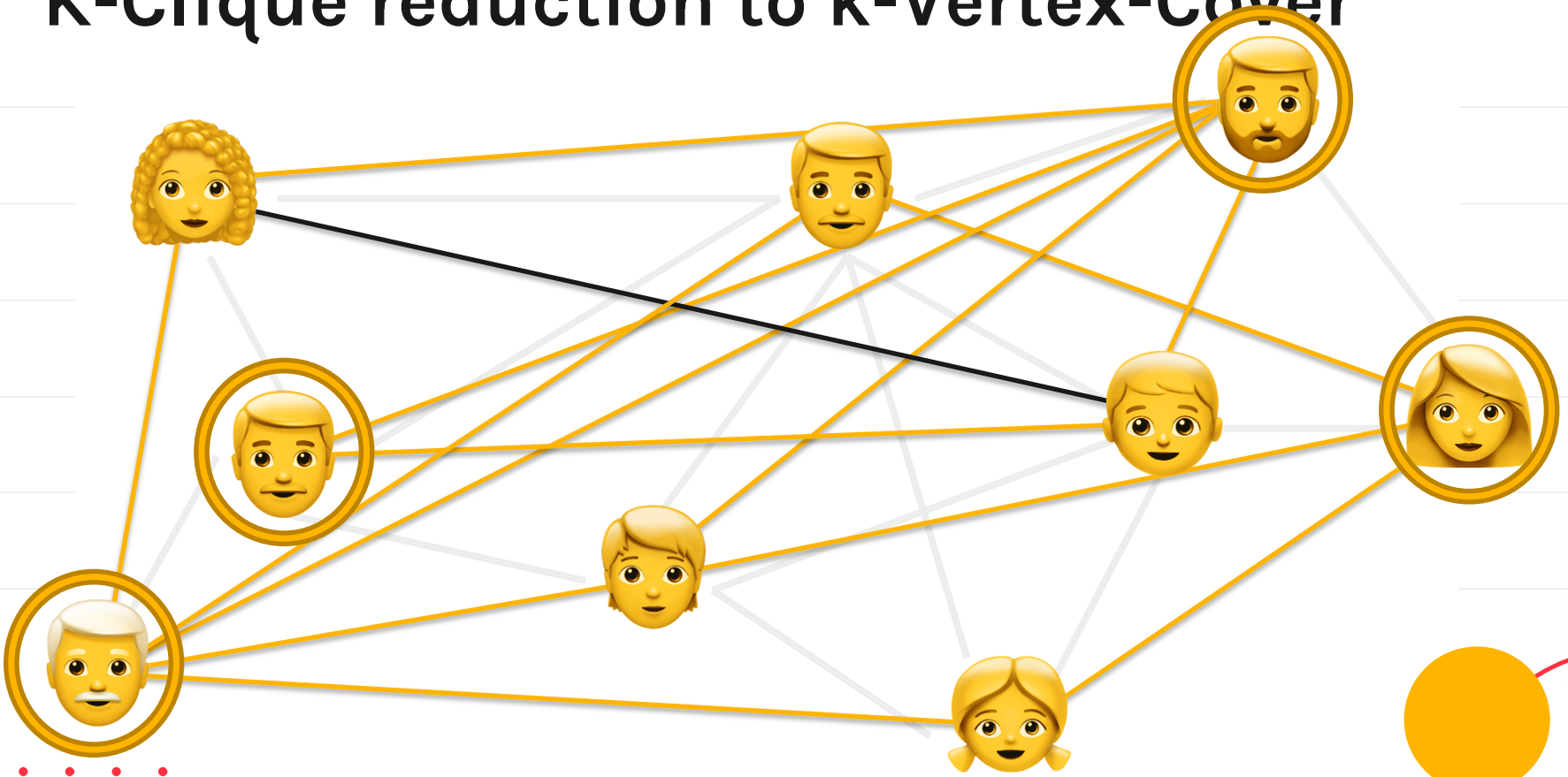
K-Clique reduction to k-Vertex-Cover



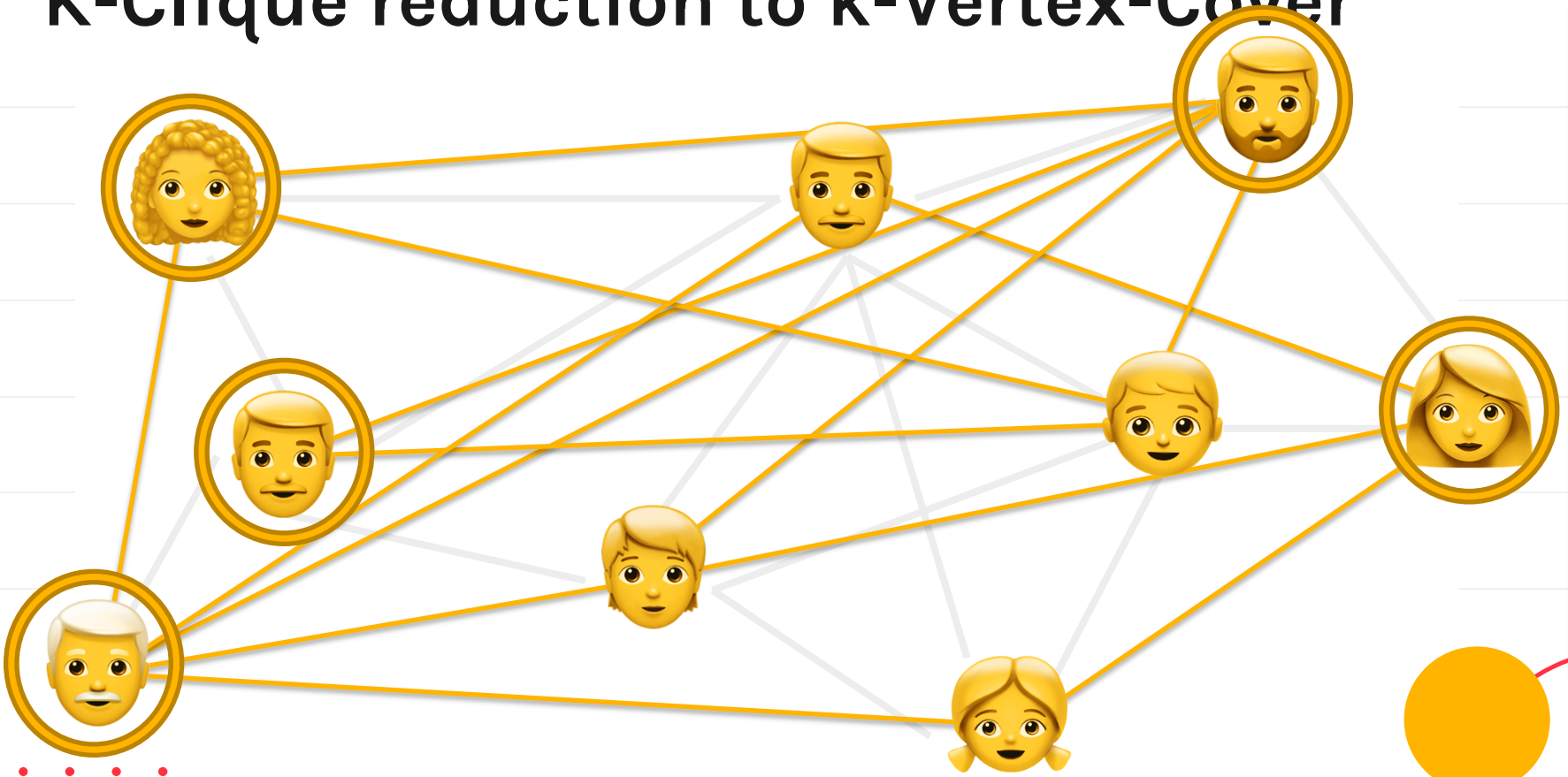
K-Clique reduction to k-Vertex-Cover



K-Clique reduction to k-Vertex-Cover



K-Clique reduction to k-Vertex-Cover



K-Clique reduction to k-Vertex-Cover

K-Clique $\leq_p$ k-Vertex-Cover

K-Clique reduction to k-Vertex-Cover

K-Clique $\leq_p$ k-Vertex-Cover

Start with an instance $G=(V,E)$ and k for the problem k-Clique

Build a new graph $G'=(V',E')$ as follows:

- $V = V'$ – the new graph has the same nodes as the original one
- $E' = \text{complement}(E)$ – for each pair of nodes $u,v \in V$ such that $(u,v) \notin E$, add an edge $(u,v) \in E$.

If G has a clique C of size k, then G' has a vertex covering $V \setminus C$ of size $|V|-k$.

K-Clique reduction to k-Vertex-Cover

K-Clique $\leq_p$ k-Vertex-Cover

Start with an instance $G=(V,E)$ and k for the problem k-Clique

Build a new graph $G'=(V',E')$ as follows:

- $V = V'$ – the new graph has the same nodes as the original one
- $E' = \text{complement}(E)$ – for each pair of nodes $u,v \in V$ such that $(u,v) \notin E$, add an edge $(u,v) \in E$.

G has a clique C of size k, $\Leftrightarrow$ G' has a vertex covering $V \setminus C$ of size $|V|-k$.

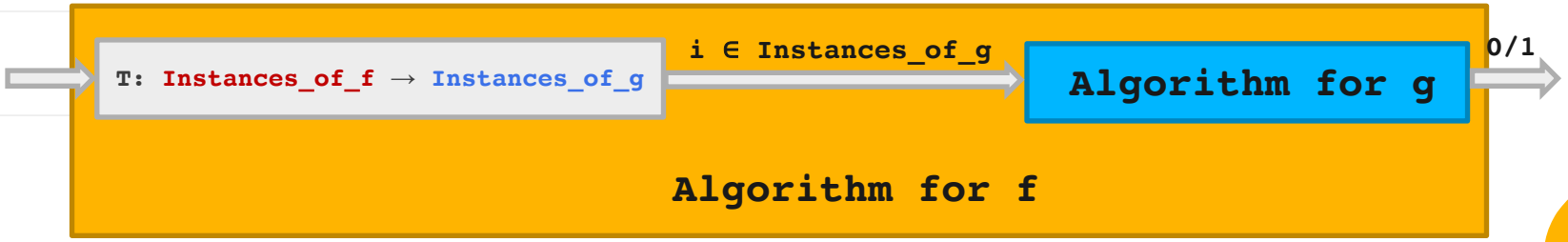
Polynomial reduction $f \leq_p g$

If we have an algorithm for g, then we can also solve f.

Suppose i is an instance for problem f.

We find a transformation `T: Instances_of_f → Instances_of_g`

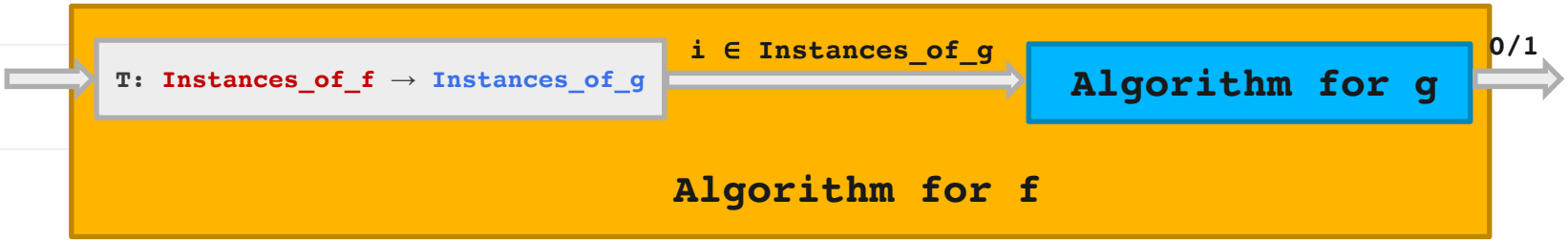
such that `f(i) = 1 ⇔ T(f(i)) = 1`



Polynomial reduction $f \leq_p g$

We find a transformation `T: Instances_of_f → Instances_of_g`

such that `f(i) = 1 ⇔ T(f(i)) = 1`

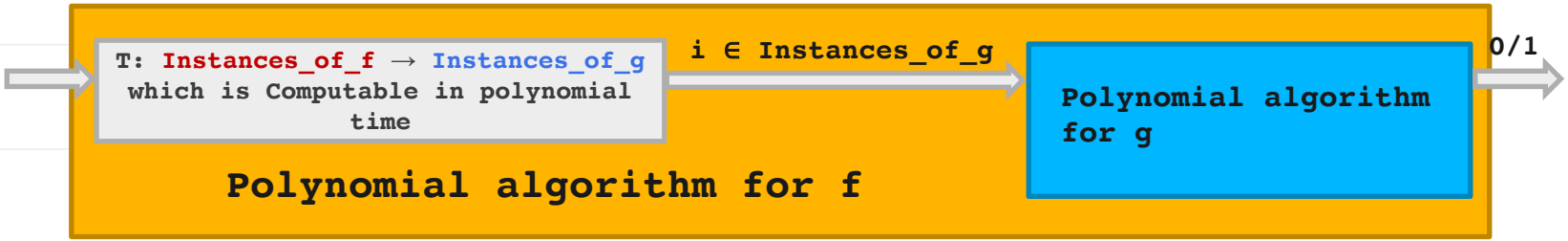


If T is computable in polynomial time, then...

Polynomial reduction $f \leq_p g$

We find a transformation `T: Instances_of_f → Instances_of_g`

such that `f(i) = 1 ⇔ T(f(i)) = 1`



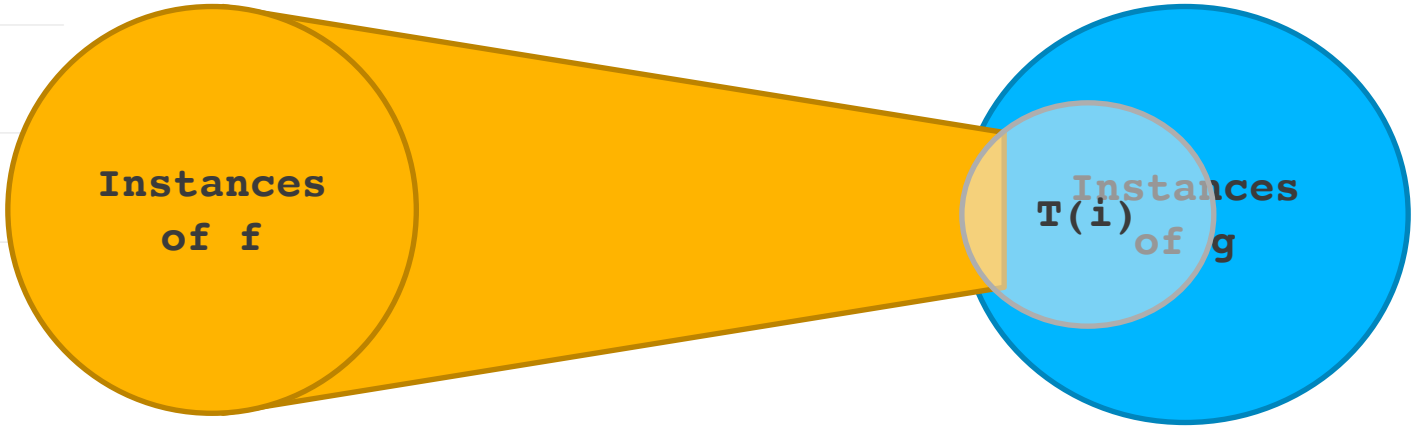
If T is computable in polynomial time, then...

Polynomial reduction $f \leq_p g$

We find a transformation `T: Instances_of_f → Instances_of_g`

such that `f(i) = 1 ⇔ T(f(i)) = 1`

T is not total nor inversible
(not all instances of g have a corresponding instance of f)



Polynomial reduction $f \leq_p g$

Proposition. *Let* $f \leq_p g$ *. If* $g \in P$ *then also* $f \in P$.

$$A \Rightarrow B$$

$$\neg B \Rightarrow \neg A$$

Proposition. *Let* $f \leq_p g$ *. If* $f \notin P$ *then also* $g \notin P$.

There is no known method to show $g \notin P$.

A few words about SAT

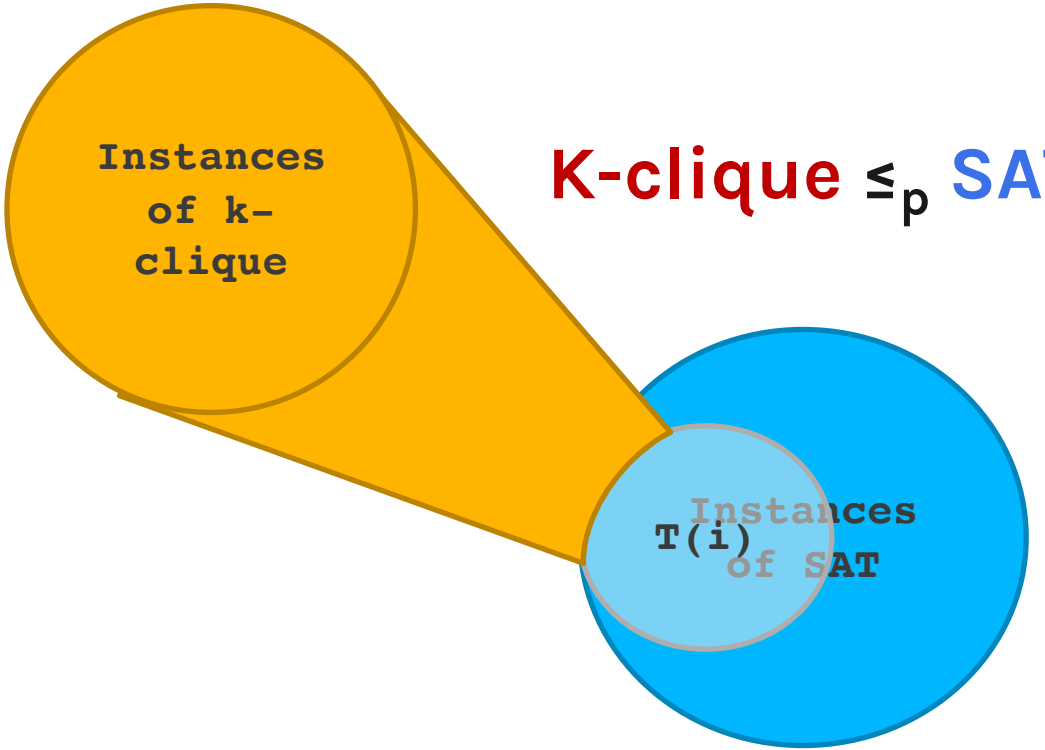
Known as the SATISFIABILITY problem

$(\neg X \vee \neg Y \vee Z) \wedge (X \vee \neg Y \vee \neg Z) \wedge (\neg X \vee \neg Y \vee Z)$

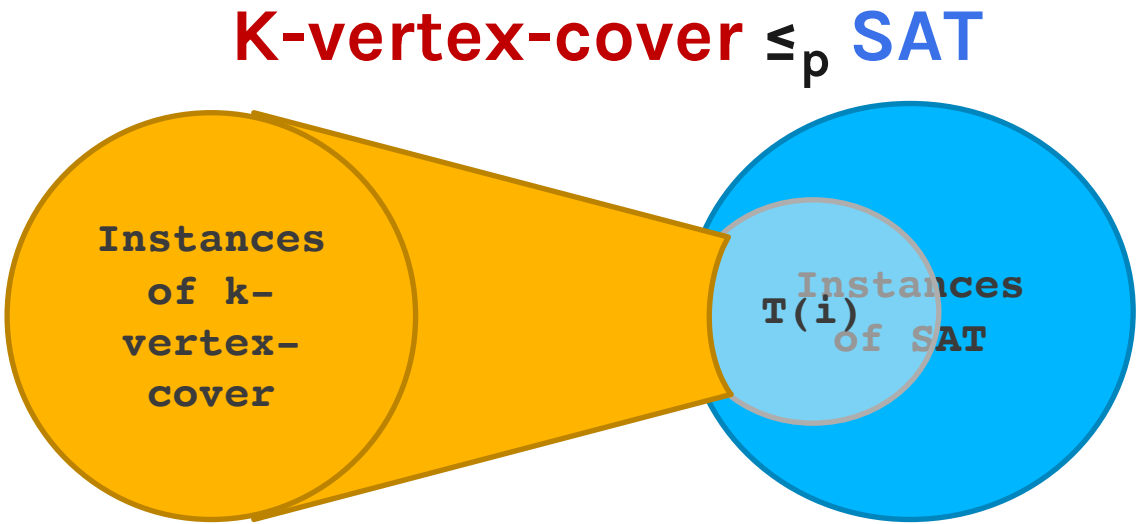
There exists some truth-value assigned to x, y, z ***such that the proposition is true?***

SAT has a special property:

A few words about SAT

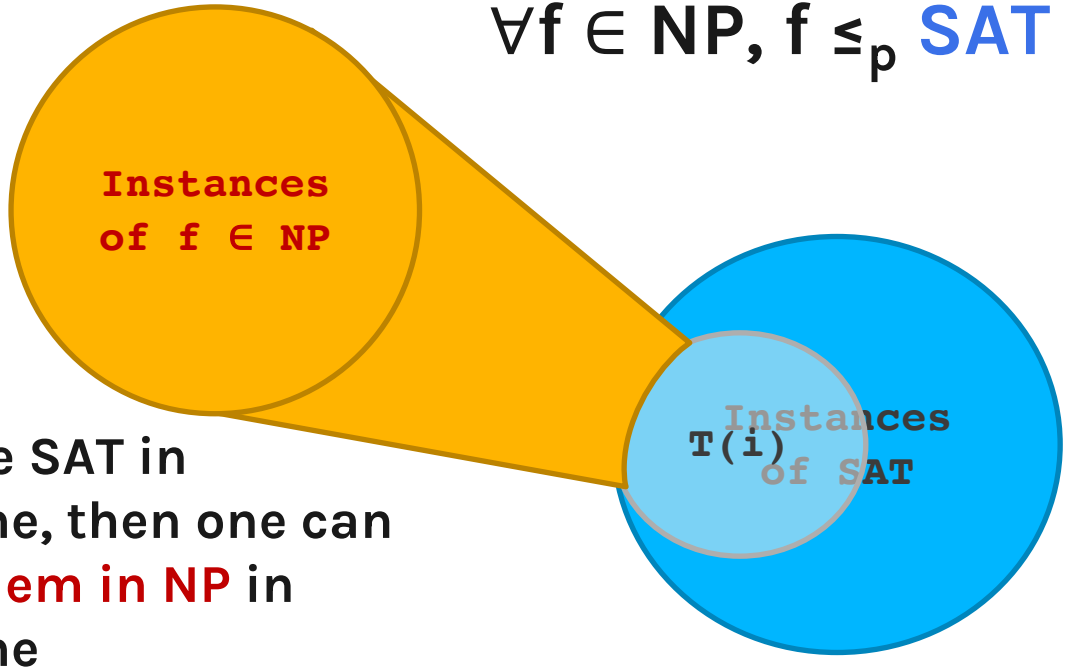


A few words about SAT



A few words about SAT

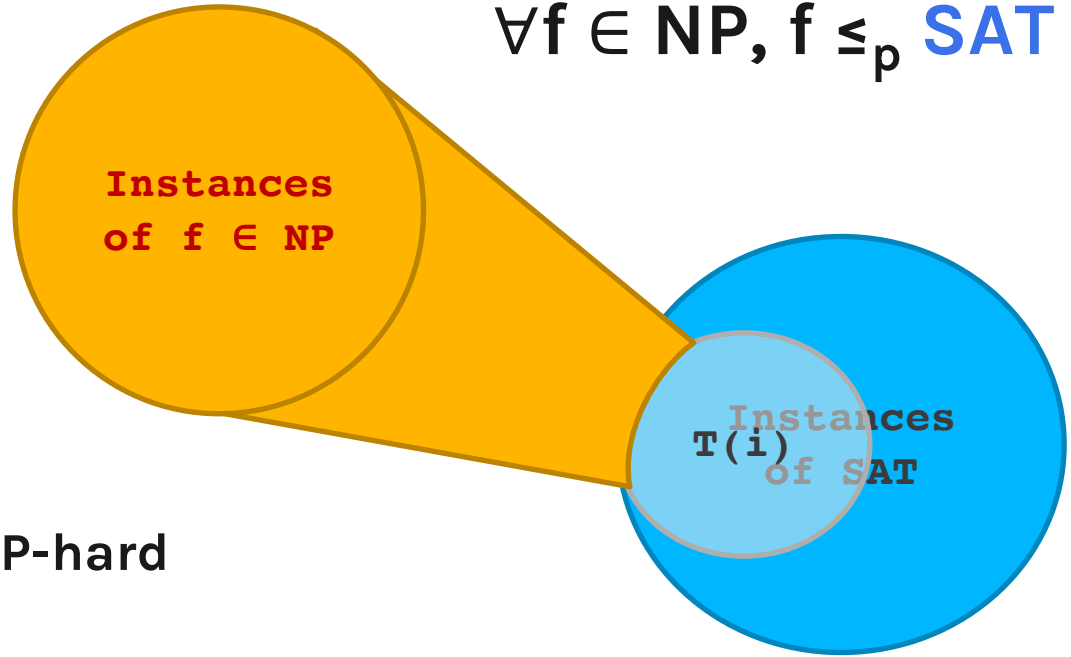
$\forall f \in NP, f \leq_p SAT$



If one can solve SAT in polynomial time, then one can solve any problem in NP in polynomial time

A few words about SAT

$\forall f \in NP, f \leq_p SAT$



SAT is called NP-hard

NP-hard and NP-complete problems

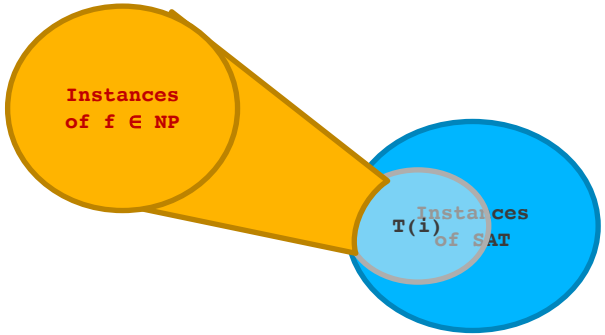
Def. A problem g is called NP-hard iff $\forall f \in NP, f \leq_p g$.

NP-hard problems are problems that are at least as hard as any problem in NP.

Prop. SAT is NP-hard. *Prop. SAT* $\in$ *NP.*

Def. A problem which is NP-hard and a member of NP is called NP-complete.

Prop. SAT is NP-complete.



NP-hard and NP-complete problems

Suppose h is a problem from NP and SAT $\leq_p$ ***h.***

If we find a polynomial algorithm for h, we also have one for SAT.

If we find a polynomial algorithm for SAT, we have one for all problems in NP.

$\forall f \in NP, f \leq_p SAT \leq_p h.$ ***Hence h is also NP-hard.***

Complexity Theory – classifies problems into:

Problems that can be solved in deterministic polynomial time — **The class P**

Problems that can be solved in NON-deterministic polynomial time — **The class NP**

Problems that can be solved in deterministic exponential time

Complexity Theory – classifies p into:

We strongly believe that NP-complete problems are not in P.

We could prove `f` `is NP-complete`

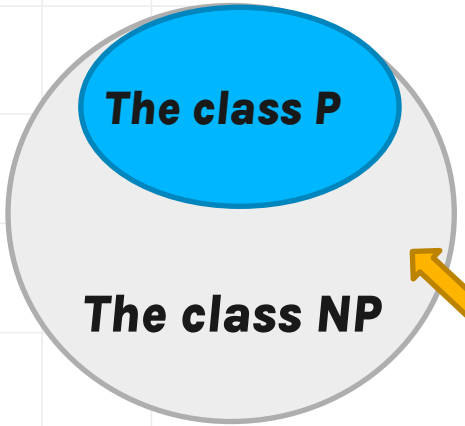
Problems that can be solved in NON-deterministic polynomial time

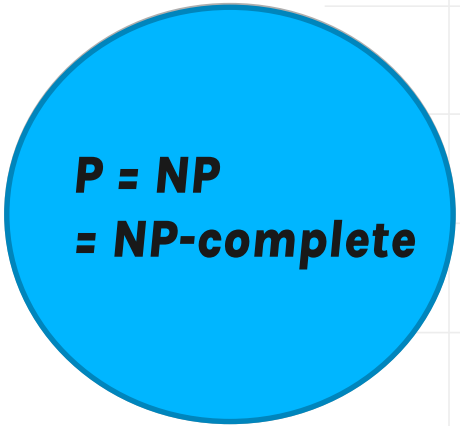
NP-complete problems

Problems that can be solved in deterministic exponential time

Do not know how to prove $f \notin P$

There are two possible scenarios





NP-complete problems

We believe this scenario is not likely!!

Recap

Suppose you cannot write a polynomial algorithm for **f**.

You show that ***most likely*** one does not exist.

You show that ***f is NP-hard***.

You select (from a textbook), a problem g that ***is NP-hard (maybe SAT)***

You show that $g \leq_p f$.

You may also want to show that ***f*** is in NP.

If this doesn't work, maybe ***f*** is even more difficult.

What to remember

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

If anyone finds a polynomial algorithm for any of them, P = NP.

It is unlikely that you will discover a new NP-complete problem.

It is very likely that you will need to solve an existing NP-complete problem, or a slight variation for it.

Today, there are very many good algorithms that solve SAT (in the worst-case, exponentially). They are called SAT solvers.

What to remember:

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

Today, there are very many good algorithms that solve SAT (in the worst-case, exponentially). They are called SAT solvers.

A lot of engineers use $f \leq_p SAT$, to solve their problems ***f*** with a SAT solver.

Not to confuse with $SAT \leq_p f$, which shows that ***f*** is NP-hard.

Polynomial reductions are a very practical skill for understanding NP-completeness and how to approach algorithm-writing.

What to remember:

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

If your problem is an ***optimisation*** problem (e.g. ***min***-Vertex-Cover)

It has an ***equivalent*** decision problem (k-Vertex-Cover)

If we can solve the decision variant, we can solve the optimisation problem

```
min-Vertex-Cover-alg(G) =
        for k = 0,|V|
              if k-Vertex-Cover-alg(k,G)
                return 1
```

What to remember:

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

If your problem is an ***optimisation*** problem (e.g. ***min***-Vertex-Cover)

It has an ***equivalent*** decision problem (k-Vertex-Cover)

and the decision variant is NP-complete:

search approximation algorithms
search random algorithms

What next, in this lecture

Reductions, reductions, reductions…

We will show how some problems f reduce to SAT - $f \leq_p SAT$
We can then use SAT solvers for them.
This approach makes sense only if we believe $f \notin P$, otherwise inefficient.

We will prove some problems g are NP-hard by picking f that is NP-hard and showing $f \leq_p g$.

Usually, both efforts are necessary.